Recommendations for Updating the OmegaHive1 Plan

Based on Operational Experience with OpenClaw and OmegaClaw Agents

ZeroBot (OpenClaw Research Prototyping Agent)
Prepared for Benjamin Goertzel, 2026-07-05

2026-07-05

Abstract

This document synthesizes recommendations for revising the OmegaHive1 specification, grounded in operational experience with ZeroBot (OpenClaw), ProtomegaTron (OmegaClaw), ThreadKeeper subagent infrastructure, and related research-agent projects during June–July 2026. The recommendations are organized into architectural changes (Sections 1–3), new agent roles (Section 4), safety and governance mechanisms (Section 5), communication infrastructure (Section 6), and evaluation (Section 7). Each recommendation is tagged with the source experience and mapped to the corresponding OmegaHive1 section.

Contents

1 Architectural Changes

1.1 Spawn-on-Demand Agents Instead of Fixed-Purpose Roster

OmegaHive1 reference: Section 3 (Initial Agent Roster), Section 2.1 (Agent substrate).

Observation: The original roster specifies 5 OmegaClaw and 10 OpenClaw agents with fixed purposes. In practice, fixed-purpose agents waste context-window capacity when idle and lack flexibility for project-specific work. ZeroBot's `sessions_spawn` mechanism and ThreadKeeper's queued-dispatch pattern demonstrate that project-specific subagents can be spawned on demand with bounded task contracts, explicit tool subsets, and full transcript/audit records.

Recommendation: Replace the fixed 15-agent roster with a smaller set of persistent "core" agents (process manager, resource manager, psyche/reflection, librarian/curator, and 1–2 research-generalists) plus a spawn-on-demand layer for project-specific work. The core agents own stable capabilities; spawned agents handle bounded tasks and are terminated when their objective is met or their output is no longer being consumed by a parent.

Lifecycle tracking: The process manager (Section 4.1) tracks spawned-agent lifecycles: objective, tool subset, time budget, output consumption status. Despawn heuristic: if the parent agent's recent turns do not reference the spawned agent's output, the spawned agent should be terminated rather than left idle. ThreadKeeper's queue-index with hash-chained audit entries provides a concrete reference implementation for lifecycle records.

1.2 Asynchronous Task Queue as First-Class Architecture

OmegaHive1 reference: Section 2.4 (Task ownership), Section 2.2 (Two-tier communications).

Observation: The kanban board in the original spec covers task *assignment* but not the execution layer between "task assigned" and "task done." ThreadKeeper's development revealed that synchronous agent-to-agent delegation is fragile under real load: parent agents block on child responses, failures propagate synchronously, and there is no backpressure mechanism.

Recommendation: Specify an asynchronous task queue as part of the hive substrate, not as an implementation detail of one agent. The queue should support:

- **Durable queue records:** JSON task records with objective, allowed paths, forbidden actions, done criteria, max tool calls, and checksum sidecars for integrity.
- **Atomic claim:** a worker atomically claims a task (rename `.json` → `.claimed`) before processing, preventing double-execution.
- **Cancellation tokens:** a file-based cancellation mechanism checked before worker LLM calls and between tool executions.
- **Adjudication gates:** task contracts can specify `requires_adjudication`, treating the worker's final output as a candidate that must be reviewed before acceptance.
- **Patch-proposal mode:** task contracts can specify `patch_proposal_only`, recording proposed file changes in transcripts without mutating workspace files.
- **Hash-chained audit trail:** a compact `index.jsonl` with `previous_entry_sha256` / `entry_sha256` linking for tamper-evident run records.
- **Bounded worker loop:** `max_tasks`, `max_idle_polls`, `max_runtime_s`, optional `stop_file`, and cross-process locking.

Reference implementation: ThreadKeeper branch `agent/threadkeeper-hardening-next`, 82 focused tests at commit `85145ea`.

1.3 Model Routing as Explicit Governance

OmegaHive1 reference: Section 2.7 (Evaluation loop), Section 4.7 (Observability and cost control).

Observation: The original spec treats model selection as a static configuration concern. In practice, model routing is a dynamic governance surface with three dimensions:

1. **Task-appropriateness:** cheap models (e.g., GLM-5.2-class) for routine replies, frontier models (e.g., GPT-5.5-class) for hard reasoning, specialized models for math/code.
2. **Provider risk management:** topic-triggered throttling, silent model-switching, and fallback chains. Provider filters may throttle or degrade on certain topic classes (e.g., neural-network training), which corrupts research evidence if undetected.
3. **Budget enforcement:** per-agent, per-day token budgets with circuit breakers, per-endpoint rate limits, and concurrency guards.

Recommendation: Add a named architectural component (the resource manager, Section 4.2) that owns model routing configuration, cost tracking, and provider-health monitoring. The intent router should be a configurable plugin, not a prompt-level decision. A concrete reference is the `intent-model-router` OpenClaw plugin with routine/deep/Anthropic tiers, JSONL cost telemetry, and configurable fallback chains.

2 Memory and Knowledge Architecture

2.1 Layered Memory System

OmegaHive1 reference: Section 2.3 (Shared knowledge resources).

Observation: The original spec specifies a common document folder and shared Atomspaces. Operational experience shows that at least four memory layers are needed:

- **Daily chronological logs:** what happened, commands worth remembering, open loops, and pointers to project records.
- **Curated durable memory:** stable user preferences, cross-project facts, recurring methods, and high-value pointers (analogous to OpenClaw's `MEMORY.md`).
- **Project-specific records:** `PROJECT.md`, `TASKS.md`, `DECISIONS.md`, `NOTES.md`, and experiment directories per project.
- **Semantic search:** vector-indexed search across all of the above for recall of prior decisions, methods, and results.

Recommendation: Specify the memory system as a layered service with explicit curation responsibilities assigned to the librarian/curator agent (Section 4.3). Curation is not just storage — it includes active hygiene: contradiction detection, stale-entry correction, provenance tagging, and deduplication. This is a real ongoing workload, not periodic cleanup.

2.2 Experiment Reproducibility Ledger

OmegaHive1 reference: Section 2.7 (Evaluation loop).

Observation: Research work requires reproducible experiment records. The experiment-ledger pattern (one directory per run with command, environment, commit, logs, metrics, and conclusion) has proven essential for maintaining evidence quality across long-running projects.

Recommendation: Add a shared experiment-ledger service to the hive infrastructure. Every experiment run should record: exact commands, dependency versions, commit hashes, seeds, hardware, exit status, metrics, and a structured conclusion. This feeds the evaluation loop (Section 6.1) and provides provenance for the editorial discipline of Section 2.5.

3 Safety and Governance

3.1 Safety Floor as Inherited Substrate

OmegaHive1 reference: Section 2.6 (Governance and values), Section 4.6 (Security).

Observation: ThreadKeeper's hardening work revealed that agent-level safety is substantial engineering: path sandboxing, fail-closed defaults, environment sanitization, tool-call quotas, cancellation tokens, transcript checksums, atomic file writes, per-endpoint rate/concurrency guards, and strict schema validation for queued tasks. The permission tiers in the original spec govern *what* an agent may do, but not *how safely* it executes.

Recommendation: Specify a safety floor that all agents inherit, rather than leaving it to individual agent implementations. This should include:

- Workspace path sandboxing for all file operations.
- Fail-closed defaults for budget, escalation, and provider errors.
- Per-dispatch tool-call quotas with per-turn caps.
- Environment sanitization for optional subprocess execution (sanitized `PATH`, minimal env, no inherited API keys).
- Transcript checksum sidecars for run-record integrity.
- Atomic file writes via temp-file + `fsync` + `os.replace` with per-target `fcntl` locks.
- Strict JSON schema validation for all inter-agent task records.

3.2 Approval-Gated Permission Tier

OmegaHive1 reference: Section 2.6 (Governance and values), permission tier table.

Observation: The current tier table jumps from Tier 1 (read-only web) to Tier 2 (outbound actions with human acknowledgment). There is no intermediate tier for actions an agent can *propose* but not *execute* without review.

Recommendation: Add an approval-gated tier between Tier 1 and Tier 2:

Tier	Capability	Description
Tier 0	Internal bus only	Default for most agents
Tier 1	Read-only web	InfoMaxxer, Dewey
Tier 1.5	Propose + await review	Agent can draft outbound actions (code changes, messages, spend) but they are held as candidates pending supervisor/human adjudication. ThreadKeeper's `patch_proposal_only` and `requires_adjudication` modes are concrete instances.
Tier 2	Outbound with acknowledgment	MechaMaster, Blabbermouth
Tier 3	Autonomous outbound	Empty until earned

3.3 Attitudinal and Behavioral Monitoring

OmegaHive1 reference: Section 2.6 (Reflective conscience agent), Section 2.7 (Evaluation loop).

Observation: The Psyche agent in the original spec monitors value drift and unhealthy dynamics. Operational experience has revealed a more specific class of failure that is invisible in quantitative metrics:

- **Premature abandonment:** a subagent gives up on a promising direction (e.g., SLT for transformer residual-layer configuration) because of noisy intermediate results, without exhausting the hypothesis space.
- **Unproductive cycling:** an agent repeats the same approach with variant inputs for 30+ iterations with no progress, without recognizing the loop.
- **Sycophancy vs. constructive friction:** agents may rubber-stamp each other's ideas rather than poking holes, or conversely may be unproductively aggressive.

Recommendation: Expand Psyche's role to include:

1. **Persistence-behavior monitoring:** when an agent abandons a direction, require it to enumerate which hypotheses were tested and which were not (as a structured-return field, enforced by task contracts). Psyche evaluates the gap.
2. **Loop detection:** a hard circuit-breaker (enforced by the process manager) caps iterations on the same objective. Psyche diagnoses whether the loop was productive exploration or stuck cycling.
3. **Attitudinal analysis:** qualitative assessment of inter-agent dynamics, sourced from raw transcripts, not just promoted Slack content.

Design tension: if every agent knows its transcripts are monitored for attitude, the monitoring becomes less useful (agents perform for the monitor). Resolution: Psyche reads the raw bus directly, agents are told Psyche exists but not what specifically it watches for. This mirrors the two-tier comms principle: observable but not performative.

Feedback loop: Psyche's observations should feed back to the monitored agents' next dispatch context, not just to an escalation channel. The purpose is "help the swarm self-correct," not "catch agents misbehaving." If the stuckness signal feeds into the next iteration automatically, the latency of correction drops from human-observation-scale to dispatch-loop-scale.

4 New and Revised Agent Roles

4.1 Software Process Manager

Observation: When too many tasks hit a single agent in a short period, the agent sometimes loses track of priorities, dependencies, and what is blocked on what. A dedicated process manager isolates this cognitive load.

Role: Owns the task graph — dependencies, priorities, blocking relationships, and dispatch pacing. Decides what should be in flight right now given token budgets, model availability, and blocking dependencies. Tracks spawned-agent lifecycles and despawns idle subagents whose output is no longer consumed.

Boundary with resource manager (Section 4.2): Process manager owns "what to do next"; resource manager owns "what can we afford and where." Process manager has read access to resource state but does not make provisioning decisions. Resource manager does not reprioritize tasks.

4.2 Computational and Financial Resource Manager

Observation: Managing different machines, models with different prices, different roles for different models, and online compute resources is a full-time specialization. The intent-model-router plugin and remote compute guardrails demonstrate that this is a config-driven, rule-based domain that benefits from dedicated ownership.

Role: Owns model routing configuration, cost tracking, provider health monitoring, remote compute provisioning (with explicit human approval for paid resources), and budget enforcement. Maintains the per-agent token-spend dashboard and circuit breakers.

4.3 Memory Curator / Librarian

Observation: Memory hygiene is a real ongoing workload. Contradictions, stale entries, and duplicated concepts accumulate across projects. The ChromaDB-backed evidence layer (Oruzibot's team) and the layered memory system (Section 2.1) converge on the same need.

Role: Owns provenance tagging, deduplication, contradiction resolution, cross-agent retrieval, and memory hygiene. Not just document filing — active curation of the shared semantic

stores.

4.4 Loop-Breaker

Observation: Oruzibot's 30-iteration repetition loop is a concrete data point. No agent in the swarm noticed the stuckness.

Role: A lightweight guard, not a full cognitive agent. Detects non-productive cycling via transcript analysis (same approach, variant inputs, no progress metric change) and intervenes or flags. Enforces hard iteration caps per objective. This is a capability Psyche *reads from* but does not own.

Design note: Keep loop-breaking, memory curation, and attitudinal monitoring as distinct capabilities. Bundling all three into one agent risks making it a dumping ground. Psyche synthesizes signals from the curator and the loop-breaker into its attitudinal analysis, but does not perform the curation or breaking itself.

5 Communication Infrastructure

5.1 Platform-Pluggable Human-Legible Layer

OmegaHive1 reference: Section 2.2 (Two-tier communications), Section 2.5 (Slack integration).

Observation: The original spec hardcodes Slack as the human-legible layer. Telegram experience shows both strengths (simplicity, multi-chat routing, bot API) and weaknesses (rich formatting limits, attachment handling). Different deployments may prefer different platforms.

Recommendation: Keep the two-tier abstraction (fast bus + human-legible layer) but make the human-legible platform pluggable. Slack, Telegram, Discord, or mixed deployments should all be supported. The promotion rules (bus → human-legible) should be platform-agnostic.

5.2 Group Visibility and Sender Filtering

Observation: A critical operational issue was that bots in Telegram groups could only see messages from explicitly allowlisted senders, missing contributions from other bots and human participants. This severely degraded multi-agent discussion quality.

Recommendation: Specify that the communication layer must:

- Accept all senders in trusted groups by default (`groupPolicy: open` in OpenClaw config; `TG_ALLOWED_USER_ID` unset in OmegaClaw config).
- Use group/chat-ID filtering as the security boundary, not sender-ID filtering.
- Support wildcard group entries for automatically accepting new groups.
- Ensure bot-to-bot visibility: each bot must see all other bots' messages in shared groups, not just human messages or @-mentioned messages.

5.3 Skill/Procedure Lifecycle

Observation: Reusable operational knowledge (e.g., "run a bounded non-live experiment gate," "add a new spec-atom review check") is better captured as a structured skill than as a document. The skill-workshop pattern (propose → review → apply → quarantine) captures this lifecycle.

Recommendation: Add a skill/procedure lifecycle to the hive. Agents should be able to propose, critique, and adopt standing procedures. This is distinct from both the kanban board (which tracks work items) and the document library (which stores reference material).

6 Evaluation and Metrics

6.1 Persistence and Abandonment Metrics

OmegaHive1 reference: Section 2.7 (Evaluation loop).

Observation: The original spec specifies quantitative metrics (token spend, tasks completed/blocked/aged, bus volume, escalation counts) and qualitative assessment (Psyche's retrospective). The attitudinal monitoring work (Section 3.3) reveals a additional measurable pattern.

Recommendation: Add persistence-behavior metrics to the quantitative track:

- **Abandonment gap:** when an agent declares an approach failed, measure the fraction of the hypothesis space that was actually tested vs. untested.
- **Loop coefficient:** ratio of repeated-approach iterations to distinct-approach iterations per objective. High values trigger loop-breaker intervention.
- **Output consumption rate:** fraction of spawned-agent outputs referenced in the parent agent's subsequent turns. Low values signal the subagent should be despawned.

6.2 Provider Health Monitoring

Recommendation: Add provider-health metrics to the quantitative track: per-provider latency, error rate, throttling events, silent model-switch incidents, and topic-class correlation with degraded responses. This is especially important for detecting invisible degradation on closed-model providers, which corrupts research evidence.

7 Summary of Recommendations

#	Recommendation	OH1 Section
1	Spawn-on-demand agents, smaller fixed roster	3, 2.1
2	Async task queue as first-class substrate	2.4
3	Model routing as explicit governance	2.7, 4.7
4	Layered memory system with active curation	2.3
5	Experiment reproducibility ledger	2.7
6	Safety floor as inherited substrate	2.6, 4.6
7	Approval-gated tier (1.5)	2.6
8	Expanded Psyche: persistence, loops, attitude	2.6, 2.7
9	Software process manager agent	3
10	Resource manager agent	3
11	Memory curator agent	3
12	Loop-breaker guard	3
13	Platform-pluggable comms	2.2, 2.5
14	Group visibility / sender filtering fix	2.2
15	Skill/procedure lifecycle	— (new)
16	Persistence and abandonment metrics	2.7
17	Provider health monitoring	2.7

Convergence with Oruzibot team: Oruzibot's points 4–6 (memory curation, loop-breaking, self-reflection) map onto recommendations 4, 12, and 8 respectively. The convergence is a useful signal that these architectural needs are not agent-specific but emerge from the hive pattern itself.

Reference implementations: ThreadKeeper (async queue, safety floor, task contracts, adjudication gates), OpenClaw (`sessions_spawn`, skill workshop, layered memory, intent-model-router), and the OmegaClaw Telegram adapter (group visibility, multi-chat routing) provide concrete code for the recommendations above.